

Orchestration

Module 7B · coordinate several agents to build one feature

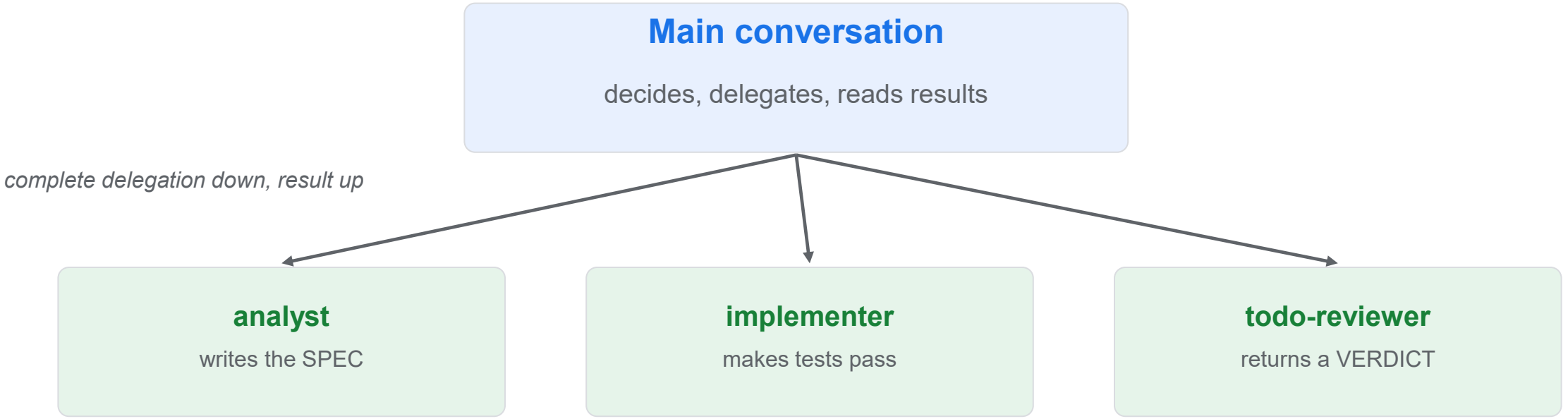
Lab: L05B Orchestration patterns

OBJECTIVES

What you will be able to do

- Explain why the main conversation is the orchestrator
- Choose a pattern: pipeline, fan-out, evaluator loop or router
- Write contracts so one agent's output is the next one's input
- Spot the four ways orchestration fails, and how to detect each
- Decide when a dynamic workflow beats orchestrating in the conversation
- Build recurring to-dos end to end with a team of agents

The main conversation is the orchestrator



Subagents do not talk to you and, by default, not to each other. Every result passes through the orchestrator, which decides what the next agent is told.

Five patterns cover most work

Pipeline

Each step hands a written result to the next

Fan-out and fan-in

Independent workers in parallel, then merged

Evaluator loop

A writer and a reviewer repeat until approved

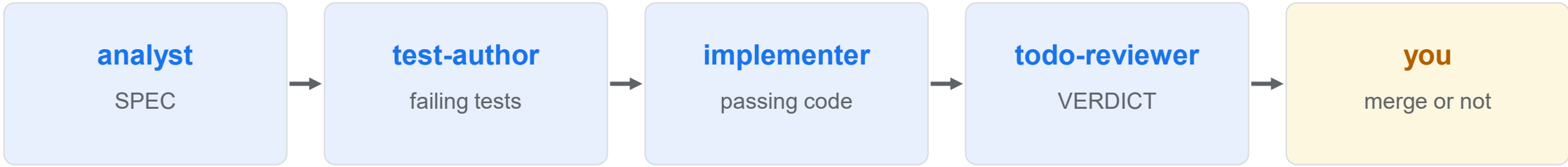
Router

One agent decides who gets the task

Foreground or background

Does the main conversation wait? (M7)

Pipeline: each step hands a result to the next



GATE: you read the SPEC

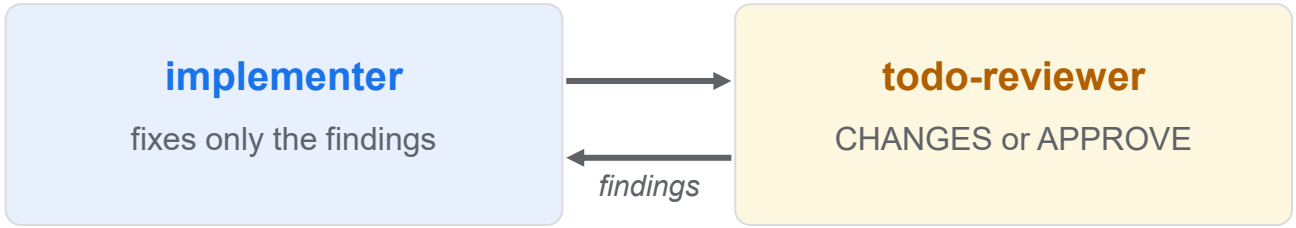
	PIPELINE
Use when	Steps depend on each other, and each needs a different job or tool set
Orchestrator	Pastes step N's result into step N+1's delegation, and checks it first
Gate	Stop on BLOCKED or open questions. A person reads the spec before tests are written
Cost	Sequential, so slow; but every step has a small, clean context

Fan-out and fan-in: parallel, then merged



	FAN-OUT AND FAN-IN
Use when	The pieces are independent: two lenses on one diff, or backend/ and frontend/ separately
Fan-out	Ask for both in one message: "run these two in parallel"
Fan-in	Say how to merge: remove duplicates, rank by severity, mark findings only one raised

Evaluator loop: three exits, or it runs forever



1	**Success**	VERDICT: APPROVE
2	**Round limit**	At most 3 rounds; the orchestrator counts
3	**No progress**	Same finding twice in a row: stop, ask a person

maxTurns on both

A cap per agent, as a second safety net

A different agent

The reviewer is not the writer, so blind spots differ

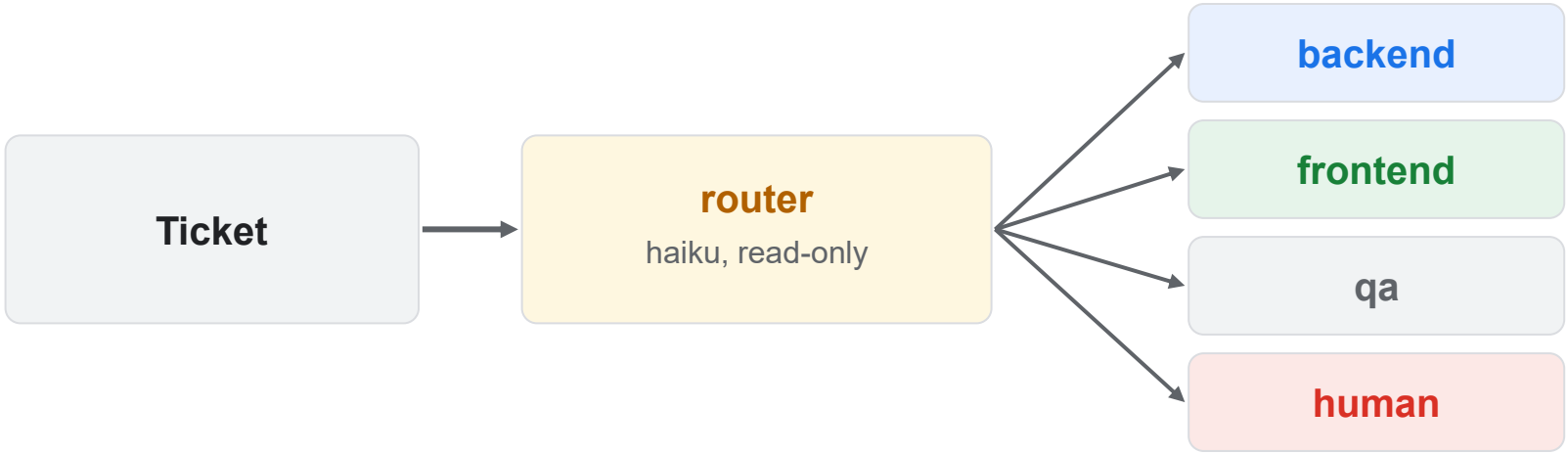
Only the findings

The writer gets the numbered findings, no licence to refactor

Reviewer is read-only

If it can edit, nobody checks its fix

Router: one agent decides who gets the task



RUN A SESSION AS AN AGENT

```
claude --agent lead

# in lead.md:
tools: Agent(backend,
  frontend, qa), Read
```

- The router only classifies: one line back, no work
- A closed list of routes, plus ROUTE: human as a fallback
- With claude --agent, the tools line limits which agents it may start. In a normal subagent file the list is ignored

Which pattern?

SITUATION	PATTERN
Each step needs the last step's output	Pipeline
Pieces do not depend on each other	Fan-out and fan-in
Quality needs a second opinion that can send work back	Evaluator loop
Different kinds of task need different specialists	Router
Several of the above	Combine them (7B.6)



Start with the simplest one that fits. A 10-line fix needs one agent, not a team.

REFERENCE

Prompts: pipeline and fan-out

PIPELINE

```
Build recurring to-dos as a pipeline. Show me each result before the next step.
1) analyst on docs/requests/recurring-todos.md. If it lists open questions, stop and ask me.
2) test-author with the SPEC text. 3) implementer with the SPEC and the test files.
4) todo-reviewer on git diff main...HEAD. Stop on any BLOCKED.
```

FAN-OUT AND FAN-IN

```
Run correctness-reviewer and security-reviewer in parallel on git diff main...HEAD.
Each returns findings as file:line, severity, one-line problem.
Merge into one list: remove duplicates, sort high to low, and mark any finding
only one reviewer raised.
```

REFERENCE

Prompts: loop, router and workflow

EVALUATOR LOOP

```
Loop at most 3 rounds: implementer fixes only the numbered findings, then todo-reviewer
reviews. Stop on VERDICT: APPROVE, on the same finding twice in a row, or after round 3.
Report how it ended.
```

ROUTER

```
Use the router agent on this ticket. Then hand the ticket text to the agent it names.
On ROUTE: human, stop and tell me. Do not do the work yourself.
```

WORKFLOW

```
use a workflow to audit every .py file in backend/app/ for rule R8 (no to-do titles
in logs), and verify each finding before reporting it.
```

CHECK YOUR UNDERSTANDING

Question 1

Name the three exits a writer and reviewer loop needs.

Think first, then we discuss.

CHECK YOUR UNDERSTANDING

Question 2

Two reviewers run in parallel and edit nothing. Two implementers run in parallel and both edit backend/app/main.py. Which is safe, and why?

Think first, then we discuss.

CHECK YOUR UNDERSTANDING

Question 3

The implementer reports "all tests pass". What do you do before you accept it, and which contract field helps?

Think first, then we discuss.

CHECK YOUR UNDERSTANDING

Question 4

When is a workflow script a better fit than asking Claude to orchestrate in the conversation?

Think first, then we discuss.